

Optimization Problem Modeling

Before you can learn a decision, you must be able to state one

Jinkyoo Park · KAIST

The atom every later method is built from

— HANDOFF

The handoff — the purest decision

Lecture 0 drew the map. We start at the corner where nothing has been taken away yet.

Where we are — the simplest cell of the map

STAGES	static ● — ○	MODEL	model-based ● — ○	AGENTS	single agent ● — ○
--------	--------------	-------	-------------------	--------	--------------------

MODEL-BASED		DATA-DRIVEN
Static, single	optimisation (Lec 1)	Bayesian / learned opt. (Lec 2–6)
Dynamic, single	MDP / optimal control (Lec 7, 9)	reinforcement learning (Lec 8, 10, 11)

We begin at the [simplest cell](#): one decision, a known objective, no time, no rivals. This is classical mathematical optimisation.

Why start here when the course is about *data-driven* decisions? Because this is the [atom](#). Every later method — Bayesian optimisation, value iteration, policy gradient, trust-region RL — is this template with something made uncertain, sequential or sampled. Master the atom and the molecules become legible.

The thesis — formulation precedes solution

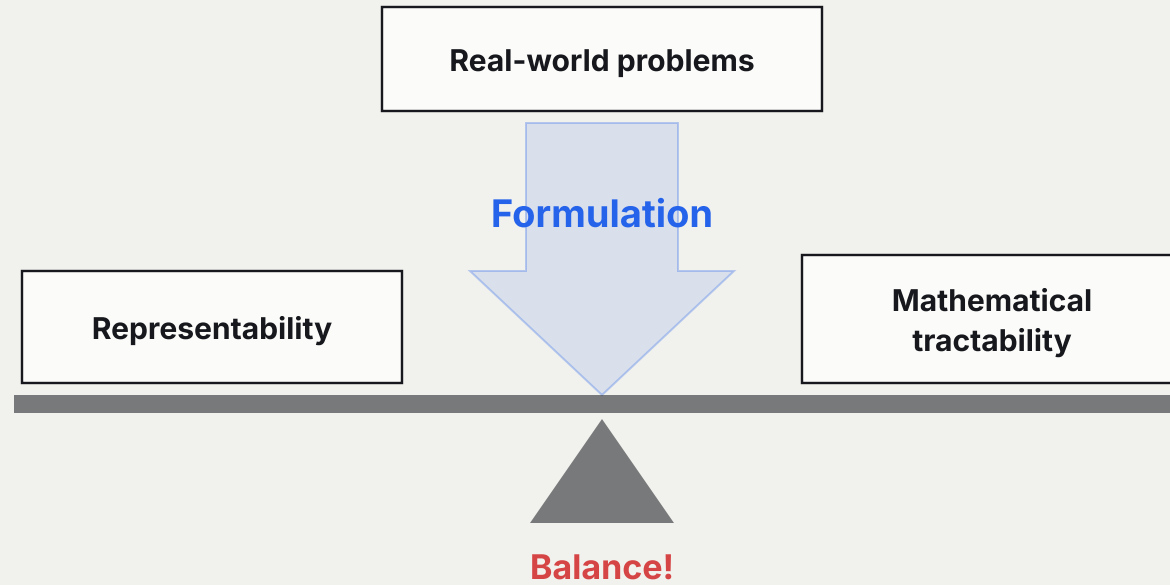
Before you can **learn** a decision, you must be able to **state** one.

A decision problem becomes mathematics through three ingredients — and naming them is half the work:

- an **objective** $f(x)$ — what "good" means, as a number to minimise;
- **decision variables** x — the levers you may pull;
- **constraints** $g(x) \leq 0, h(x) = 0$ — what is allowed.

Get this right and the rest is method. Get it wrong — the wrong objective, a missing constraint — and no solver, classical or learned, can save you. The hardest and most consequential step in applied decision making is the **modelling**, not the solving.

Formulation is a balance



A model that keeps every detail of the world cannot be solved; a model a solver loves may no longer be the problem you had. **Formulation is the act of balancing the two** — and it is where most of the real difficulty of applied decision making lives.

The roadmap — four questions



- Q1 — How do we state a decision mathematically? The [standard form](#).
- Q2 — Which problems can we actually solve? [Convexity](#) — the great watershed.
- Q3 — How do we *know* a solution is optimal? Optimality conditions and [KKT](#).
- Q4 — What about the non-convex ones? [Successive convexification](#) and trust regions.

— ACT 1

The standard form

Q1. One shape that every optimisation problem can be poured into.

The standard form — one shape for all of them

Q1 State it?

Q2 Solve it?

Q3 Certify it?

Q4 Convexify it?

Every mathematical optimisation problem can be written:

$$\min_{x \in D} f(x) \quad \text{s.t.} \quad g_i(x) \leq 0, \quad i = 1, \dots, m, \quad h_j(x) = 0, \quad j = 1, \dots, p$$

- $x \in \mathbb{R}^n$ — the optimisation variable; $f : \mathbb{R}^n \rightarrow \mathbb{R}$ — the objective;
- g_i — inequality constraints; h_j — equality constraints;
- the **optimal value** $p^* = \inf\{f(x) : x \text{ feasible}\}$.

Two degenerate cases worth naming: $p^* = +\infty$ if the problem is *infeasible* (no x satisfies the constraints), and $p^* = -\infty$ if it is *unbounded below*. Both are usually signs of a modelling error, not a solver failure.

Maximising f is minimising $-f$; everything is phrased as minimisation without loss of generality.

— ACT 2

Convexity, the watershed

Q2. Of all the problems you can state, which can you actually solve?

Convexity — the line between easy and hard

Q1 State it?

Q2 Solve it?

Q3 Certify it?

Q4 Convexify it?

A problem is **convex** when f is a convex function and the feasible set is a convex set — g_i convex, h_j affine.

WHY CONVEXITY IS THE DIVIDING LINE

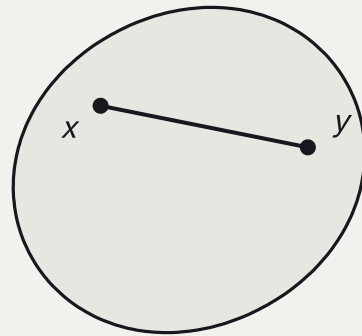
For a convex problem, **any local minimum is a global minimum**.

That single fact changes everything:

- reliable, efficient solvers exist; initialisation and step size do not trap you in bad local optima;
- global optimality is **certifiable** — via KKT, in Act 3;
- duality gives a computable lower bound and an optimality gap;
- the problem decomposes well for distributed and large-scale solving.

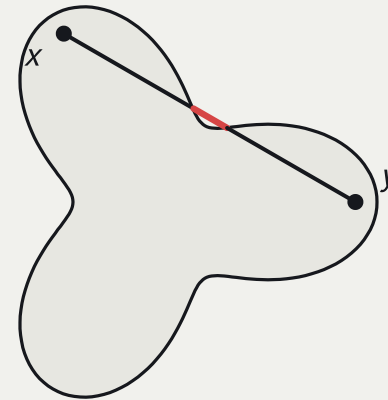
Non-convex problems enjoy none of these for free. Much of practical optimisation is the art of **getting a convex problem** — or a sequence of them.

Convex sets — the segment test



convex

every chord stays inside



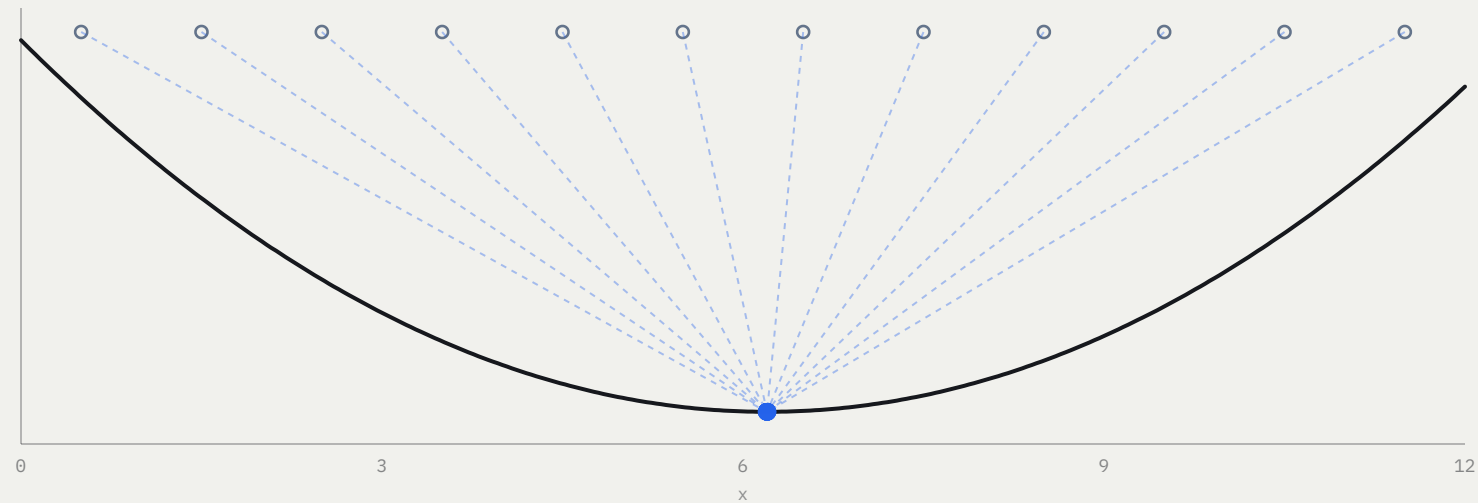
not convex

this chord leaves the set

A set is convex when the segment joining any two of its points stays inside it. A function is convex when the segment joining any two points of its graph stays *above* it — equivalently, when its epigraph is a convex set. **Both halves of a convex problem are this one test.**

The watershed, run twice

LOCAL DESCENT, TWELVE STARTS, ONE OBJECTIVE



The same descent rule, the same twelve starting points, two objectives. On the convex one the starting point is irrelevant; on the non-convex one it decides the answer. Everything in Act 2 follows from this single picture.

The convex family — LP, QP, QCQP

Named convex forms, in order of generality.

CLASS	OBJECTIVE	FEASIBLE SET
Linear program (LP)	$c^\top x + d$	a polyhedron (affine inequalities)
Quadratic program (QP)	$\frac{1}{2}x^\top Px + q^\top x$, with $P \succeq 0$	a polyhedron
QCQP	convex quadratic	an intersection of ellipsoids

The progression matters because modelling is often a matter of *recognising* which named class your problem — or its convexified version — falls into; each has mature, reliable solvers. The LP's optimum sits at a vertex of the polyhedron; the QP's, where the quadratic's level sets first touch the feasible region.

These same quadratic models reappear in Act 4 as the *local* approximation of hard problems — and, much later, as the trust-region subproblem inside TRPO (Lecture 10).

— ACT 3

Certifying optimality

Q3. A solver returns a point. How do you know it is *the* point?

When is a point optimal? — the first-order condition

Q1 State it?

Q2 Solve it?

Q3 **Certify it?**

Q4 Convexify it?

For a convex problem $\min_{x \in D} f(x)$ with differentiable f , a feasible x^* is optimal **if and only if**

$$\nabla f(x^*)^\top (y - x^*) \geq 0 \quad \text{for all feasible } y$$

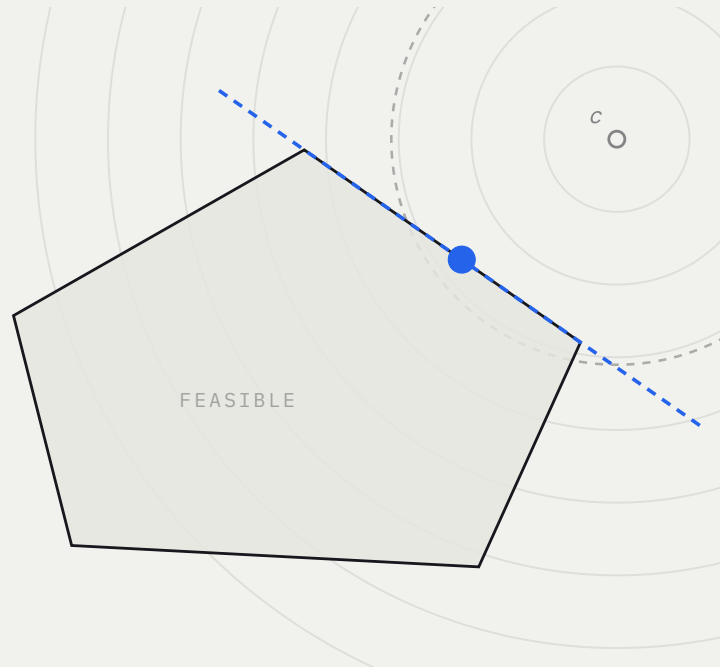
Reading it geometrically. Moving from x^* toward any other feasible point cannot decrease f — there is no improving feasible direction. If $\nabla f(x^*) \neq 0$ it defines a **supporting hyperplane** to the feasible set at x^* : the whole feasible region lies on the uphill side.

Optimality = **no feasible direction points downhill.**

For an *unconstrained* convex problem this collapses to the familiar $\nabla f(x^*) = 0$.

The condition, made draggable

DRAW THE POINT — IS ANY FEASIBLE DIRECTION DOWNHILL?



OPTIMAL

no feasible direction points downhill

$$x = (7.47, 6.65) \cdot f(x) = 9.63$$

$$\min_{y \text{ feasible}} \nabla f(x)^T (y - x) = \mathbf{0.00} \geq 0$$

The unconstrained minimiser sits outside the feasible set, so the optimum is pushed onto the boundary. There the gradient is normal to the active edge and points out of the set — that normal, scaled, is the multiplier $\lambda \geq 0$ of KKT.

Minimise $\|x - c\|^2$ over a polygon, with c outside it. Drag the point and read the test literally: the red arrow is a feasible direction that decreases f . It disappears exactly at the optimum, where the gradient supports the set — and that supporting normal, scaled, **is the KKT multiplier**.

KKT — optimality with constraints, and a dual bound

With constraints, introduce multipliers $\lambda_i \geq 0$ for g_i and ν_j for h_j . The **Karush–Kuhn–Tucker** conditions characterise optimality:

$$\nabla f(x^*) + \sum_i \lambda_i \nabla g_i(x^*) + \sum_j \nu_j \nabla h_j(x^*) = 0, \quad g_i(x^*) \leq 0, \quad h_j(x^*) = 0, \quad \lambda_i \geq 0, \quad \lambda_i g_i(x^*) = 0$$

The last identity, **complementary slackness**, says each constraint is either active ($g_i = 0$) or ignored ($\lambda_i = 0$). For convex problems KKT is *necessary and sufficient* — a certificate of global optimality.

Duality, in one line. The Lagrangian $L(x, \lambda, \nu) = f(x) + \sum_i \lambda_i g_i + \sum_j \nu_j h_j$ yields a dual function whose value **lower-bounds** p^* for any $\lambda \geq 0$. The gap between primal and dual is a computable measure of how close you are.

Differentiating the KKT system is exactly how one back-propagates through an optimisation layer — the trick behind differentiable LQR and MPC in Lecture 11.

— ACT 4

When the problem is not convex

Q4. Real problems are not convex. Do we give up the guarantees, or manufacture them?

Successive convexification — solve a sequence of easy problems



Real engineering problems are usually non-convex — non-convex objective, non-convex constraints. The dominant strategy: **don't solve the hard problem; solve a sequence of convex approximations to it.**

At the current iterate $x^{(k)}$, build a local convex model

$$\tilde{f}(x) = f(x^{(k)}) + \nabla f(x^{(k)})^\top (x - x^{(k)}) + \frac{1}{2}(x - x^{(k)})^\top B^{(k)}(x - x^{(k)})$$

linearise the awkward constraints, and add a **trust region** $\|x - x^{(k)}\| \leq \rho^{(k)}$ so the model stays trustworthy.

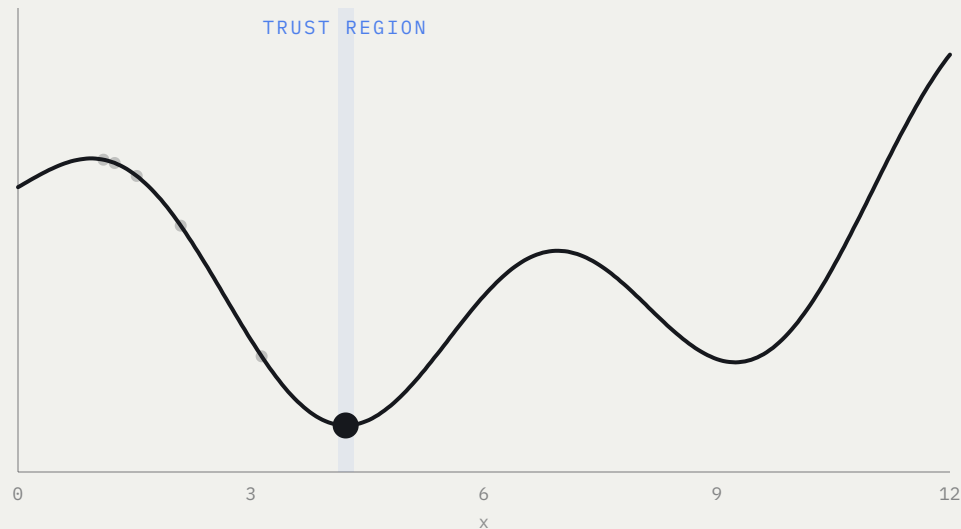
The trust region is the key idea: **only trust the approximation nearby.**

Solve the convex subproblem; if it improves the true objective, accept and *grow* ρ ; if not, reject and *shrink* ρ . Repeat.

The trust region, iterated

SUCCESSIVE CONVEXIFICATION — A STAIRCASE OF EASY PROBLEMS

ITERATE 15

**CONVERGED**

the gradient has vanished and the trust region has closed in

$$x^{(k)} = 4.219 \cdot f = 0.864$$

trust radius $\rho = 0.100$ · gradient $f' = -0.000$

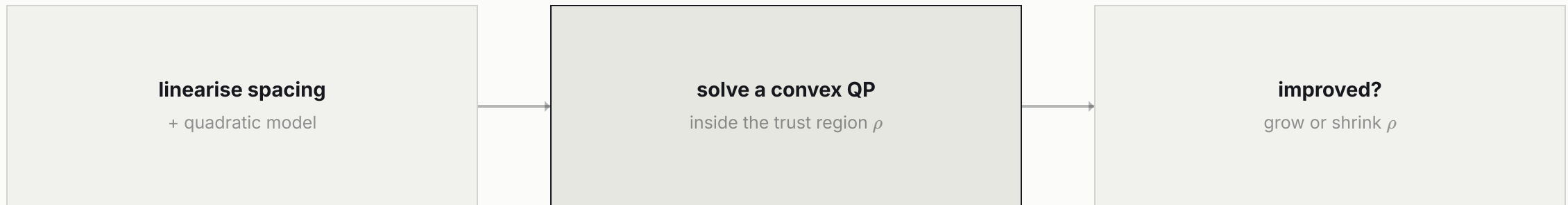
accepted steps: 10 of 15

The ratio asks whether the convex model deserved to be believed. Trust grows where it predicts well and shrinks where it does not — nothing about the true objective is ever solved directly.

Step it and watch the ratio test do the work: an accepted step earns more trust, a rejected one halves it. The true objective is never solved — only a staircase of quadratics inside a shrinking and growing box. [This exact logic returns as TRPO's KL trust region in Lecture 10.](#)

Case study — wind-farm layout optimisation

Place N turbines to maximise power, subject to minimum inter-turbine spacing and boundary constraints — a non-convex problem, because the spacing constraints are non-convex.



Each iteration is a convex QP, solved reliably; the trust region keeps the linearised constraints close to the true ones. The non-convex layout problem is conquered by **a staircase of convex problems** — the practical face of Act 2's lesson: *get a convex problem, even if you have to keep making new ones.*

— CLOSING

Closing

We can state a decision and, when it is convex, prove we have solved it. Now the assumption we drop.

Where we are — and the assumption we now drop

MODEL-BASED		DATA-DRIVEN
Static, single	optimisation (Lec 1 ✓)	Bayesian / learned opt. (Lec 2–6 →)

We can now *state* a decision and, when it is convex or convexifiable, *solve and certify* it. But every line above assumed one thing:

the objective f and the constraints were known, **exactly**.

What if f has *uncertain parameters* — a model fitted to noisy data? Then a single best x is not enough; we must reason about our **uncertainty about f itself**. That is Lecture 2: don't pick a number — carry a belief.

Optimization is how a decision becomes mathematics — and, when the problem is convex, an answer we can not only find but *prove*.

An objective to minimise, levers to pull, limits to respect.

Questions?

Hold onto two pieces: the standard form — it is the skeleton of every objective to come — and the trust region, which returns in surrogate optimisation and again in trust-region RL. Everything else this term is this atom, perturbed by uncertainty.

— APPENDIX

Backup slides

Complete statements, kept out of the narrative.

Backup 1 — the KKT conditions in full

For $\min f(x)$ subject to $g_i(x) \leq 0$ and $h_j(x) = 0$, the KKT conditions at x^* , with multipliers $\lambda^* \geq 0$ and ν^* :

-
- **Stationarity** — $\nabla f(x^*) + \sum_i \lambda_i^* \nabla g_i(x^*) + \sum_j \nu_j^* \nabla h_j(x^*) = 0$
 - **Primal feasibility** — $g_i(x^*) \leq 0, h_j(x^*) = 0$
 - **Dual feasibility** — $\lambda_i^* \geq 0$
 - **Complementary slackness** — $\lambda_i^* g_i(x^*) = 0$
-

Convex case. If f, g_i are convex and h_j affine, and a constraint qualification such as Slater's holds, KKT is *necessary and sufficient* for global optimality — a checkable certificate. **Non-convex case.** KKT remains necessary under a constraint qualification but not sufficient: a KKT point may be a local minimum, a saddle, or a local maximum.

Backup 2 — Lagrangian duality and the gap

Lagrangian. $L(x, \lambda, \nu) = f(x) + \sum_i \lambda_i g_i(x) + \sum_j \nu_j h_j(x)$.

Dual function. $d(\lambda, \nu) = \inf_x L(x, \lambda, \nu)$. For any $\lambda \geq 0$ and any ν , weak duality gives $d(\lambda, \nu) \leq p^*$ — the dual is a *lower bound* on the optimum, always.

Dual problem. $\max_{\lambda \geq 0, \nu} d(\lambda, \nu) =: d^*$. The **duality gap** is $p^* - d^* \geq 0$. For convex problems with a constraint qualification the gap is zero ($p^* = d^*$, *strong duality*), so the dual optimum certifies the primal.

Use. Even when you cannot solve the primal, evaluating $d(\lambda, \nu)$ at any feasible dual point brackets how far your current solution can possibly be from optimal — the practical value of duality.

Backup 3 — the successive convex programming loop

Algorithm — trust-region SCP.

1. Initialise $x^{(0)}$ and a trust radius $\rho^{(0)}$.
2. **Repeat** until $\|x^{(k)} - x^{(k-1)}\| < \epsilon$:
3. form the convex model $\tilde{f}$ — gradient plus an approximate Hessian $B^{(k)}$ — and linearise the constraints;
4. solve the convex subproblem over $\{x : \|x - x^{(k)}\| \leq \rho^{(k)}\}$, giving a candidate $\tilde{x}$;
5. compute the ratio $r = \frac{f(x^{(k)}) - f(\tilde{x})}{f(x^{(k)}) - \tilde{f}(\tilde{x})}$ — actual over predicted improvement;
6. if $r \geq a$: accept $x^{(k+1)} = \tilde{x}$ and grow $\rho^{(k+1)} = \beta_{\text{succ}} \rho^{(k)}$;
7. else: reject $x^{(k+1)} = x^{(k)}$ and shrink $\rho^{(k+1)} = \beta_{\text{fail}} \rho^{(k)}$.

Why the ratio test. It measures whether the convex model can be trusted at the proposed step. Trust grows where the model predicts well and shrinks where it does not — the exact logic that, in policy space, becomes TRPO's KL trust region in Lecture 10.